

RAPPORT QUALITÉ **MY PULSE**

SAE 3.05



PRÉSENTÉ PAR :
HOJA VALENTIN & SCHMITT LILIAN





SOMMAIRE

1. ETAT ACTUEL DU PROJET

- Portée et objectifs
- Stack technique
- Structure du projet

2. ANALYSE DE LA DOCUMENTATION

- Lacune critique identifiée
- Éléments de documentation positifs

3. ANALYSE DE L'APPROCHE QUALITÉ

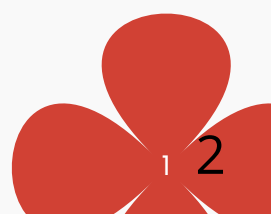
- Cadre de QA actuel
- Risques identifiés

4. RECOMMANDATIONS DES OUTILS

- Outils recommandés

5. CONCLUSION

6. RÉFÉRENCES





1. ETAT ACTUEL DU PROJET

1.1 PORTÉE ET OBJECTIFS

MyPulse est une application web full-stack conçue pour :

- Permettre aux utilisateurs de voter sur des morceaux musicaux, des artistes et des groupes dans une liste partagée
- Afficher en temps réel le classement des contenus basé sur les votes
- Fournir un dashboard administrateur pour la gestion des sessions de vote

1.2 STACK TECHNIQUE

Composant	Technologie
Backend Base de données Frontend Serveur Web Contrôle de version	PHP 8.3.28 MySQL HTML5, CSS3, JavaScript Apache 2.4.65 / Wampserver Git / GitHub

1.3 STRUCTURE DU PROJET

Le projet est organisé selon une architecture MVC standard :

class/ - Classes PHP

controllers/ - Contrôleurs

create/ - Création de contenu

 |--- uploads/ - Uploads utilisateurs

database/ - Fichier de configuration (script sql)

fonts/ - Polices

icons/ - Icônes

index/ - Pages principales

login/ - Authentification

pages/ - Pages légales

script/ - JavaScript

style/ - CSS

vote/ - Système de vote

 |---script/ - JavaScript



2. ANALYSE DE LA DOCUMENTATION

2.1 LACUNE CRITIQUE IDENTIFIÉE

2.2.1 Absence de Guide Utilisateur

Les utilisateurs finaux ne disposent pas de documentation explicite sur :

- Comment créer une session de vote
- Comment voter sur des morceaux
- Comment accéder au dashboard administrateur
- Résolution des problèmes courants

2.2.2 Processus d'Installation Non Standardisé

Il n'existe pas de documentation claire des étapes pour :

- Cloner le projet
- Configurer les variables d'environnement (.env)
- Initialiser la base de données

2.3 ÉLÉMENTS DE DOCUMENTATION POSITIFS

- Utilisation de contrôle de version Git avec historique commit cohérent
- Fichier .env pour la configuration d'environnement
- Structure de projet clairement organisée
- Code source logiquement nommé et structuré





3. ANALYSE DE L'APPROCHE QUALITÉ

3.1 CADRE DE QA ACTUEL

Actuellement, MyPulse opère sans processus de QA formalisé :

Points positifs :

- Développement local sur Wampserver permettant l'isolation
- Utilisation de Git pour le versioning et les rollback
- Architecture modulaire facilitant les tests unitaires

Lacunes critiques :

- Aucun test automatisé (unitaires, intégration, fonctionnels)
- Pas de pipeline CI/CD formalisé
- Absence de checklist de test avant déploiement
- Pas de métriques de qualité (couverture de code, etc.)
- Absence de documentation des bugs/incidents
- Pas de stratégie de versioning sémantique

3.2 RISQUES IDENTIFIÉS

Risque	Impact	Probabilité	Criticité
Bug en production non détecté	Élevé	Moyenne	Critique
Régression après modification	Élevé	Moyenne	Haute
Données utilisateur corrompues	Très élevé	Faible	Critique
Incompatibilité dépendances	Moyen	Faible	Moyenne



4. RECOMMANDATIONS DES OUTILS

OUTILS RECOMMANDÉS

Wampserver pour l'hébergement local de l'application

PHPUnit pour réaliser des tests unitaires

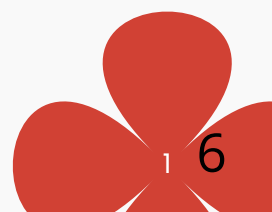
VSCode avec des plugins documentation et autocomplétion

Bootstrap ou Tailwind CSS pour incorporer des styles

Trello pour la gestion des tâches

GitHub Wiki ou GitBook pour documentation centrale

Lucidchart pour les diagrammes architecturaux





5. CONCLUSION

MyPulse dispose d'une excellente base technique et d'une architecture bien pensée, mais souffre actuellement d'une absence de documentation systématique et de processus QA formalisé. Cette lacune représente un risque croissant à mesure que le projet évolue.

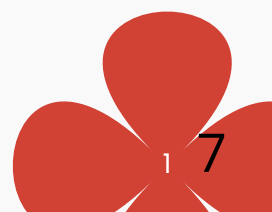
Les recommandations proposées dans ce rapport offrent un chemin clair et progressif vers un cadre qualité robuste :

Court terme (1-2 mois) : Mettre en place les fondations (CI/CD, tests unitaires, documentation) pour réduire immédiatement les risques en production.

Moyen terme (2-6 mois) : Consolider avec tests intégration, documentation utilisateur complète et métriques de qualité automatisées.

Long terme (6+ mois) : Optimiser avec tests performance, audit sécurité et monitoring avancé.

Return On Investment (ROI) attendu : Confiance accrue des utilisateurs dans la fiabilité du système (anonymisation, pas de fraudes)





6. RÉFÉRENCES

SmallTox, « Assurance qualité logiciel informatique : bonnes pratiques », 2025. Consulté le 13 janvier 2026 depuis <https://www.smalltox.com/assurance-qualite-logicielle-informatique-assurance/>

Visure Solutions, « Qu'est-ce que l'assurance qualité (AQ) : processus... », 2025. Consulté le 13 janvier 2026 depuis <https://visuresolutions.com/fr/alm-guide/quality-assurance/>

France Compétences, « RS1822 – CFTL ISTQB Testeur Certifié Niveau Fondation », 2024. Consulté le 14 janvier 2026 depuis <https://www.francecompetences.fr/recherche/rs/1822/>

Atlassian. (2025). Software Documentation Best Practices + Real Examples. Consulté le 8 janvier 2026 depuis <https://www.atlassian.com/blog/loom/software-documentation-best-practices>

TestDevLab. (2025). Best Practices for Effective Document Management in Quality Assurance. Consulté le 11 janvier 2026 depuis <https://www.testdevlab.com/blog/document-management-in-software-qa>